AlgoAscent: Binary Search Master Class

Competitive Programming: From Basics to Advanced Optimization

Materials Society

December 11, 2025

"The Art of Elimination: Stop Scanning, Start Dividing."

The Intuition: Linear vs. Binary

The "Guess a Number" Game (Range 1 to 100)

Linear Search (The "Toddler" Way)

- "Is it 1? Is it 2? ... Is it 99?"
- We eliminate **1 option** at a time.
- **Worst Case:** 100 Guesses ($O(N)$).
- *Inefficient for large datasets.*

Binary Search (The "Pro" Way)

- "Is it 50?" $\rightarrow$ "Too High" (Eliminates 51-100).
- "Is it 25?" $\rightarrow$ "Too Low" (Eliminates 1-24).
- We eliminate **half the universe** each step.
- **Worst Case:** ≈ 7 Guesses ($\log_2 N$).

CRITICAL PREREQUISITE

Binary Search ONLY works if the search space is **SORTED** or **MONOTONIC**.

Template 1: The Standard Implementation

This is the standard iterative approach used for finding an exact value in a sorted array.

```cpp
// Standard Template for finding 'target' index
int binarySearch(vector<int>& arr, int target) {
    int left = 0;
    int right = arr.size() - 1;

    while (left <= right) {
        // Prevent Integer Overflow: DO NOT USE (left + right) / 2
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid; // Found it!
        } else if (arr[mid] < target) {
            left = mid + 1; // Target is in the right half
        } else {
            right = mid - 1; // Target is in the left half
        }
    }
    return -1; // Not found
```

Implementation Details: The Overflow Bug

The Bug

Many beginners write: `int mid = (left + right) / 2;`
If `left` and `right` are large (e.g., $2 \cdot 10^9$), their sum exceeds the integer limit ($2^{31} - 1$), causing a negative overflow.

The Fix: Mathematically, $\frac{L+R}{2}$ is the same as $L + \frac{R-L}{2}$, but the second form prevents the sum from ever exceeding the maximum value of R.

```
// ALWAYS USE THIS FORMULA
int mid = left + (right - left) / 2;
```

Time & Space Complexity: The 10^8 Rule

Why Logarithms Rule Competitive Programming
In most contests (Codeforces, LeetCode), the time limit is 1 second ($\approx 10^8$ operations).

Input Size (N)	**Linear $O(N)$**	**Binary $O(\log N)$**	**Verdict**
10	10 ops	4 ops	Same
10^5 (100k)	100,000 ops	17 ops	Huge Gap
10^9 (1 Billion)	**TLE (10 sec)**	**30 ops**	**INSTANT**

Space Complexity:

- **Iterative:** $O(1)$ (Best for CP).
- **Recursive:** $O(\log N)$ (Due to stack memory).

STL Mastery: Don't Reinvent the Wheel

C++ provides highly optimized binary search functions in `<algorithm>`.

- `binary_search(begin, end, val)`: Returns `true` if found.
- `lower_bound(begin, end, val)`: Returns iterator to first element $\geq$ *val*.
- `upper_bound(begin, end, val)`: Returns iterator to first element $>$ *val*.

```
vector<int> v = {10, 20, 30, 40, 50};

// Find first element >= 35
auto it = lower_bound(v.begin(), v.end(), 35);
cout << *it; // Prints 40

// Find first element > 35
auto it2 = upper_bound(v.begin(), v.end(), 35);
cout << *it2; // Prints 40

// Count occurrences of 30
int count = upper_bound(...) - lower_bound(...);
```

Template 2: The Invariant (Mastery)

Use Case: Finding First/Last occurrence, or when boundaries are tricky.
Concept: Maintain `L` in the "False" zone and `R` in the "True" zone. Converge until they are neighbors.

```cpp
// Find the first index where arr[i] >= K
int solve(vector<int>& arr, int K) {
    int l = -1;              // Always False zone
    int r = arr.size();      // Always True zone

    while (r - l > 1) {      // Stop when they are adjacent
        int mid = l + (r - l) / 2;
        if (arr[mid] >= K) {
            r = mid;         // mid is True -> shrink Right
        } else {
            l = mid;         // mid is False -> shrink Left
        }
    }
    return r; // r is the first element >= K
}
```

Advanced Paradigm: Search on Answer

Problem Type: "Minimize the Maximum" or "Maximize the Minimum".
Example: Koko Eating Bananas (LeetCode 875).

The Concept:

- Instead of searching an array index, we search the **Range of Possible Answers**.
- If the answer is "Speed", the range is $[1, 10^9]$.
- **Monotonicity:** If speed K works, then speed $K+1$ DEFINITELY works. We want the *smallest* K.

`[ F, F, F, F, T, T, T, T ]`
Binary Search finds the boundary between F and T.

Search on Answer: Implementation

```cpp
bool check(int speed, int h, vector<int>& piles) {
    long long hours = 0;
    for(int p : piles) hours += (p + speed - 1) / speed; // ceil
    return hours <= h;
}

int minEatingSpeed(vector<int>& piles, int h) {
    int low = 1, high = 1e9;
    int ans = high;

    while(low <= high) {
        int mid = low + (high - low) / 2;
        if(check(mid, h, piles)) {
            ans = mid;      // Valid! Try slower speed?
            high = mid - 1;
        } else {
            low = mid + 1;  // Too slow! Need faster.
        }
    }
```

Advanced: Rotated Sorted Arrays

Problem: Array is sorted but shifted: `[4, 5, 6, 7, 0, 1, 2]`.
Logic: If you split it, **one half is always sorted**.

```
if (nums[low] <= nums[mid]) { // LEFT is sorted
    if (target >= nums[low] && target < nums[mid]) {
        high = mid - 1; // Target is in left
    } else {
        low = mid + 1;  // Target is in right
    }
}
else { // RIGHT is sorted
    if (target > nums[mid] && target <= nums[high]) {
        low = mid + 1;  // Target is in right
    } else {
        high = mid - 1; // Target is in left
    }
}
```

Advanced: 2D Matrix Flattening

Problem: Search in an $M \times N$ matrix where rows are sorted and flow into the next.
Trick: Treat the 2D grid as a 1D array of size $M \times N$.

Coordinate Mapping:

- Index range: $[0, M \cdot N - 1]$
- `Row = mid / cols`
- `Col = mid % cols`

```
int low = 0, high = (rows * cols) - 1;
while(low <= high) {
    int mid = low + (high - low)/2;
    int val = matrix[mid / cols][mid % cols];

    if (val == target) return true;
    else if (val < target) low = mid + 1;
    else high = mid - 1;
}
```

Mastery: Continuous & Meta Search

Continuous Search (Geometry/Math)

- Used for finding roots ($x^2 = N$).
- Do not use `while(l <= r)`.
- Use fixed iterations for precision.

```
for(int i=0; i<100; i++) {
    double mid = (l+r)/2;
    if(check(mid)) r = mid;
    else l = mid;
}
```

Meta Search (Powers of 2)

- Used for LCA (Binary Lifting) or Unbounded Arrays.
- Jump by $2^k, 2^{k-1} \dots 1$.

```
int curr = -1;
for(int step=n/2; step>=1; step/=2) {
    while(curr+step < n && arr[curr+
        step] <= val) {
        curr += step;
    }
}
```

Summary: The Cheat Sheet

Problem Requirement	Technique	Complexity
Check existence	Standard Template	$O(\log N)$
First/Last Index	Invariant Template	$O(\log N)$
STL Shortcut	`lower_bound`	$O(\log N)$
Min Capacity/Time	Search on Answer	$O(\log(\text{Range}))$
Rotated Array	Pivot Logic	$O(\log N)$
Real Numbers	Continuous (100 iters)	$O(1)$

Final Tip: When in doubt, use the **Invariant Template** (`r - l > 1`).
It is harder to mess up boundary conditions with it.